Service Discovery Pattern

Category: Communication **One-liner:** A registry that lets services find each other by *name* at runtime, instead of hardcoding network addresses that change constantly.

Part 0 – Concepts you need first

What is an IP address and a port?

An **IP address** identifies a machine on a network (e.g. `10.0.3.47`). A **port** identifies which program on that machine (e.g. `8080`). Together – `10.0.3.47:8080` – they identify one running service instance.

To call another service, you must know its address and port. That's the entire problem this pattern exists to solve: in modern deployments, **those numbers change all the time**.

What is an "instance"?

One running copy of a service. If your Order Service is busy, you run five copies to share the load – five **instances**, each with its own IP address, all running identical code.

This is **horizontal scaling**: handle more load by adding more copies, rather than by making one copy bigger.

What is a container, and why does it make addresses unstable?

A **container** packages an application with everything it needs to run, so it starts anywhere in seconds. **Kubernetes** is the standard system for running containers across a fleet of machines.

The key consequence: containers are **disposable**. Kubernetes starts them, stops them, moves them between machines, and replaces them during deployments. **Each new one typically gets a new IP address.** An address you looked up ten minutes ago may now point at nothing.

What is auto-scaling?

Automatically adding instances when load rises and removing them when it falls. AWS auto-scaling groups do this. It saves money – but it means **the number of instances and their addresses change throughout the day without anyone touching anything**.

What is load balancing?

Distributing incoming requests across multiple instances so no single one is overwhelmed. If you have five instances and one hundred requests, ideally each handles about twenty.

What is DNS?

The **Domain Name System** translates names into IP addresses – you type `example.com`, DNS returns the address. It's the internet's original name-to-address lookup, and some service discovery systems (notably Kubernetes) work by extending exactly this idea inside your cluster.

Part 1 – The problem

In a **monolith**, components are tightly integrated and deployed as one unit. Component A calling component B is just a **method call inside the same process** – no network, no address, nothing to discover. Service discovery is a non-problem.

In **microservices**, services are loosely coupled and deployed independently. They may live on different servers, in different data centres, even on different continents. Now every call is a network call to an address that

must be known.

Three forces make hardcoded addresses unworkable.

1. Dynamic infrastructure

Modern deployments – virtual machines, containers, Kubernetes – give services **no fixed IPs or hosts**. Instances come and go, scale up and down, and get replaced during deployments.

The course's example: an AWS auto-scaling group launches new instances of a service or terminates them based on load, **each time with new IP addresses**. Keeping track of these moving targets is a serious challenge.

2. Undetermined number of instances

Multiple instances of the same service run for load balancing. So: **which instance should a client call?**

- Hardcoding one address means all traffic goes to that one instance – you've lost load balancing entirely, and if it dies, everything fails
- Maintaining a list by hand is error-prone, and becomes untenable as services and instances multiply

3. Coupling

A goal of microservices is that teams deploy and scale independently. But if every client must know every service's exact address, you have **tight coupling**. A location change means updating configuration in **every client that calls it**.

Deployment details leak into client configuration, and the agility you adopted microservices for disappears.

The concrete scenario

An online retail application. A user places an order:

- **Order Service** must call **Inventory Service** to check item availability
- Then call **Payment Service** to process payment

How does Order Service know where those are? What happens when Inventory moves to a new server? What happens when extra Payment instances spin up under load – how does Order Service learn they exist?

Part 2 – The solution

Introduce a **central registry and lookup mechanism**.

Dynamic registration

Each service instance, **when it starts, registers its address (host and port) with a central Service Registry**. When shutting down, it de-registers. The registry therefore always holds a current listing of active instances. Clients no longer need hardcoded addresses.

Lookup by logical name

Services register under a **logical name** – `OrderService`, `UserService`. A client needing to call a service simply asks the registry for the address of `"OrderService"`, and gets back the current location, or several if multiple instances exist.

> **This indirection is the entire trick.** Instance addresses can change freely because clients only ever know the *name*. It's the same idea as a phone book: you look up a person by name, and it doesn't matter that their number changed last year.

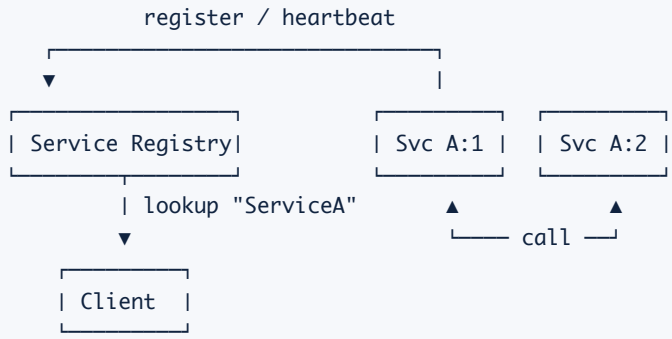
Load balancing and scaling

With multiple instances running, discovery works together with a load-balancing strategy. Scaling horizontally becomes trivial: **register more instances, and clients discover them automatically**.

Decoupled configuration

Clients maintain no configuration of service endpoints, staying decoupled from other services' deployment details. This makes continuous deployment and auto-scaling possible without disrupting clients. The client supplies a service name; **actual routing happens at runtime**.

Part 3 – Architecture



The Service Registry

A database holding all services and their current locations – the **centralized source of truth**, effectively a phone book for your system.

Three responsibilities:

1. Service Registration. When a service comes online it registers itself, providing its name, IP address, port, and potentially metadata such as service version or health endpoint.

2. Service Deregistration. Removing a service – either when it shuts down gracefully, or when the registry detects it's no longer available.

3. Health Checking. The registry periodically pings services or their health endpoints to check they're still running. If a service doesn't respond within a certain timeframe, it's considered failed and removed.

> *Why health checking is essential:* graceful deregistration only covers graceful shutdown. If an instance **crashes**, loses power, or gets partitioned off the network, it never gets the chance to say goodbye. Without health checks the registry would hand out dead addresses forever.

The payoff: the system **adapts dynamically**. A service moves or scales → the registry updates → others still find it. A service dies → it's removed → others can apply fallback strategies.

Part 4 – Client-side vs. server-side discovery

Both use a registry. They differ in **who performs the lookup and load balancing** – and this is the single most likely thing you'll be asked about in an interview.

Client-side discovery – "smart client"

The workflow:

1. **Service Registration** – each instance registers with the registry at startup, providing its address and perhaps a health heartbeat; it unregisters on shutdown. 2. **Service Lookup** – when a client needs to call a

service, it **queries the registry directly** for a list of active instances. 3. **Client-side Load Balancing** – the client (or a library on the client side) selects one of the instances, often using round-robin or consistent hashing. 4. **Request Invocation** – the client makes the request **directly to the chosen instance**. Subsequent requests may repeat the lookup or use a cached result.

The client is **"smart"**: it contains logic to query the registry and balance load.

The classic stack: Netflix **Eureka** as the registry, combined with a client load-balancer library such as **Ribbon** or **Spring Cloud LoadBalancer**. The client knows the Eureka location, uses it to find instances, then calls one directly.

Advantages:

- **No extra network hop** – after discovery the client talks to the service directly
- **Intelligent decisions** – because the client knows all instances, it can implement sophisticated, application-specific load balancing (e.g. choose an instance based on request content)
- **Fewer moving parts at runtime** – aside from the registry, no additional routing layer is needed
- **Slightly lower latency** – no intermediate proxy

Disadvantages:

- **Every client needs the discovery logic and library**, increasing complexity
- **Painful in polyglot environments** – with services in multiple languages you need a compatible discovery client for each, or a standardized approach. This is a real barrier
- **Two network calls per request** – one to the registry, then the actual request (though caching usually eliminates the first)
- **Couples clients to the discovery mechanism** – if the registry changes, many clients may need updating, which is a maintenance burden

The standard mitigation: clients **cache** registry responses. This avoids a lookup on every call *and* lets the system keep working if the registry is temporarily down.

Server-side discovery – "smart router, dumb client"

The workflow:

1. **Service Registration** – identical; instances register at startup, deregister on shutdown. 2. **Request to Load Balancer** – the client sends its request to a **known, fixed endpoint** – a load balancer, gateway, DNS name, or virtual IP. **The client never queries the registry.** 3. **Service Discovery & Routing** – the load balancer receives the request and queries the registry for healthy instances. 4. **Load Balancing** – the load balancer selects an instance using its own algorithm. 5. **Forwarding** – it forwards the request to the chosen instance. From the client's perspective it simply made a request to a single endpoint; it has no idea any of this happened.

The intermediary is "smart" and the client is simple.

Advantages:

- **Much simpler clients** – one call to a fixed address, no discovery code, no special libraries
- **Language-agnostic** – any client that can make a normal HTTP request works, because discovery happens in the network infrastructure. This is the decisive advantage in a polyglot estate
- **One network call per request** – no separate lookup
- **Abstraction** – where services actually run can change without impacting clients
- **Often provided out of the box** – Kubernetes and cloud load balancers give you this for free

Disadvantages:

- **An extra hop** through the load balancer, adding slight latency
- **The load balancer must be highly available and scalable** – it's now critical to every request
- **Operational overhead** if your environment doesn't already provide one
- **The client loses control** over which instance it talks to – it can't tailor routing per request and must trust the router's decisions

Choosing between them

	Client-side	Server-side
Who queries the registry	The client	The load balancer / gateway
Network hops	Fewer (direct call)	One more (through the router)
Client complexity	Higher – needs a library	Very low
Polyglot friendliness	Poor	Excellent
Routing control	Fine-grained, app-specific	Whatever the router does
Typical stack	Eureka + Ribbon (Spring Cloud)	Kubernetes Services, cloud LBs

Practical guidance:

- In the **Spring Cloud / Netflix OSS** world, client-side (Eureka + Ribbon) is straightforward
- On **Kubernetes**, the platform's built-in discovery via DNS means you lean server-side – you simply call `http://my-service` and Kubernetes routes it
- **They're not mutually exclusive** – you might use client-side for internal service-to-service calls while having an API gateway (server-side style) for external traffic

In modern systems – Kubernetes, service meshes – **server-side discovery is the norm**, because the platform abstracts discovery behind DNS or a gateway.

Part 5 – Inner workings

Self-registration vs. third-party registration

Self-registration: services register and deregister *themselves* when they come online or go offline.

- *Upside:* simplicity, and the service has full control
- *Downside:* the service carries the added responsibility of keeping its status up to date

Third-party registration: a **service manager** handles registration. It monitors services and updates their status in the registry.

- *Upside:* takes the burden off the services entirely
- *Downside:* introduces another component that must itself be managed

Health checks – two models

Self-reporting (the heartbeat model). Services send a periodic signal to the registry saying they're still active. If the registry doesn't receive a signal within a given timeframe, it assumes the service is offline and removes it.

Active monitoring. The registry proactively checks services by sending a ping or similar request. No response → marked offline.

> The difference matters under load: heartbeats put work on the services and scale with instance count; active monitoring puts work on the registry and scales with *its* capacity to poll everyone.

Part 6 – Worked example (simplified Java)

The registry – a phone book

```
import java.util.concurrent.ConcurrentHashMap;

public class ServiceRegistry {
    private ConcurrentHashMap<String, String> services = new ConcurrentHashMap<>();

    public void register(String serviceName, String serviceAddress) {
        services.put(serviceName, serviceAddress);
    }

    public String discover(String serviceName) {
        return services.get(serviceName);
    }
}
```

Reading it: keys are service names, values are addresses. `register` adds a service; `discover` retrieves an address by name.

Why `ConcurrentHashMap`? Registration and lookup happen from many threads simultaneously — many services registering while many clients look things up. A plain `HashMap` would corrupt under concurrent modification. `ConcurrentHashMap` is designed for exactly this.

A service registering itself

```
public class Service {
    private String name;
    private String address;

    public void register(ServiceRegistry registry) {
        registry.register(name, address);
    }
}
```

This is **self-registration**. When the service starts it calls `register` to add itself. In a more sophisticated system this happens automatically — via a service manager, or as part of the deployment process.

A client discovering and calling

```
public class ClientService {
    private ServiceRegistry registry;

    public void callService(String serviceName) {
        String serviceAddress = registry.discover(serviceName);

        if (serviceAddress == null) {
            throw new RuntimeException("Service not found: " + serviceName);
        }

        // Make a request to the service
    }
}
```

The client uses the registry to find the address of the service it wants. If not found, it throws. If found, it proceeds with the request.

What this toy version is missing

The course is explicit that real implementations are more complex. Specifically:

- **Health checks** — services must regularly report in to keep their registration current

- **A distributed registry** – to handle high traffic and provide redundancy
- **Failure handling in** `callService` – retrying the request, or falling back to a backup service
- **A *list* of instances, not one address – `Map<String, String>` stores exactly one address per service name, so this design** cannot load balance at all**. A real registry stores `Map<String, List<String>>` and returns all healthy instances so the client can choose

That last point is the most important gap to notice – it's the difference between a demo and something usable.

Part 7 – Performance implications and special considerations

1. Latency overhead

Every registry lookup is an extra step. In client-side discovery this is usually small because clients **cache** – fetching the instance list once and reusing it for many requests, refreshing every ~30 seconds or on a cache miss. The lookup is normally a fast local network call taking milliseconds.

To minimise latency:

- Cache and reuse discovery information for a short duration rather than querying per request
- Design the registry to respond quickly (in-memory indexes)
- Use asynchronous discovery updates – pushed updates or long-polling
- In high-performance scenarios, bake a small local endpoint cache into each instance at startup to avoid even the initial lookup delay

2. Scalability of the registry

It must handle registrations from potentially thousands of instances plus frequent lookups. It can become a bottleneck.

- **Clustering** – run the registry as a cluster of multiple nodes (e.g. three instances of Eureka or Consul) sharing load and providing redundancy. Configure clients with multiple registry URLs
- **Sharding** – partition registries by service domain. Uncommon; one cluster usually suffices for a large organisation
- **Efficient heartbeats** – thousands of instances heartbeating every 30 seconds is genuine load. Eureka lets you tune the heartbeat interval and use **self-preservation mode** to avoid overwhelming the network during an outage or partition
- **Lightweight protocols** – Consul uses an efficient **gossip** mechanism for health; Zookeeper uses an event-driven model with persistent connections

3. Fault tolerance and high availability

If the registry is down, services may be unable to find each other – especially with client-side discovery.

- **Run multiple registry instances** – at least two or three in production. Keep data in sync via consensus (Consul, Zookeeper) or peer-to-peer sync (Eureka)
- **Client-side caching for resilience** – many clients continue using the last known good information if the registry becomes unreachable. Spring's Eureka client does this by default, allowing the system to keep functioning assuming topology hasn't changed drastically
- **Failover of registry nodes** – configure clients with multiple registry addresses so they try the next when one fails
- **Monitor the registry itself**, not just the services it tracks
- **Data persistence** – Consul, etcd, and Zookeeper persist to disk and recover state after restart. Eureka keeps state in memory and relies on clients re-registering. Persistence helps recovery if all nodes reboot, but may be overkill if services simply re-register on startup

Consistency vs. availability – the CAP trade-off

> *Beginner note – CAP theorem:* in a distributed system, when a network partition splits your nodes apart, you must choose between **consistency** (never serve wrong data, even if that means serving nothing) and **availability** (always answer, even if the answer may be stale). You cannot have both during a partition.

	Zookeeper / Consul	Eureka
Choice	**CP** – consistency	**AP** – availability
Behaviour	Avoids serving possibly-stale data; may become unavailable until consensus is restored	Continues serving whatever it has, even if stale
On partition	May refuse to answer	**Self-preservation** – won't purge entries, so some info stays available

Be aware of your registry's behaviour and tune it. For example, turn **off** self-preservation in small setups to get quicker removal of dead instances.

> *Which is right?* For service discovery, **availability usually beats consistency**. A slightly stale instance list is survivable – you try an address, it fails, you retry another. A registry that refuses to answer means nothing can talk to anything. This is why Eureka chose AP.

4. Load balancing strategies

- **Round Robin** – each new request goes to the next instance cyclically. Distributes evenly assuming instances have similar capacity
- **Random** – pick randomly each time; roughly balances over many requests and is trivially simple
- **Least Connections / Load** – track which instance has fewest active requests and send there, balancing *actual* load. Requires more sophisticated tracking; usually done in server-side LBs or service meshes
- **Hashing (sticky sessions)** – consistently send the same client to the same instance, for caching or session affinity. E.g. hash of user ID modulo instance count
- **Geographic / AZ affinity** – in multi-datacenter or multi-availability-zone deployments, prefer an instance in the client's zone for lower latency. Discovery metadata carries the zone

Ribbon defaults to round-robin on the client side; a cloud ELB might do round-robin or least connections by configuration.

5. Failover handling

What happens when a call to an instance fails? With discovery you have multiple instances, so try another.

- **Client retry** – on timeout or error, query the registry again (or use the cached list), pick another instance, and retry. Ribbon and Spring Cloud LoadBalancer have retry mechanisms
- **Circuit breakers** – Hystrix or Resilience4j can trip if a service fails repeatedly and stop calling it for a cooldown. Combined with discovery this can be **per-instance**: if instance X of Service A is failing, stop sending to X while continuing with Y and Z. Discovery is what ensures Y and Z are available *(see Circuit Breaker)*
- **Health-based removal** – the registry should promptly mark failing instances unhealthy so lookups stop returning them. Proactive, and better than reacting at request time
- **Timeout tuning** – keep timeouts short. Fail fast and try another instance; that's viable precisely because you have a discovered pool

6. Consistency of data

The registry's view of which instances are up can lag reality. If a service crashes there's a window before the registry notices (the next heartbeat period), during which a client may be handed an instance that just died. To minimise impact:

- **Reduce heartbeat intervals** – faster detection, at the cost of more network chatter

- **Client pre-flight health check** before critical requests – adds overhead, rarely done
- **Tolerate the failed attempt and retry another instance** – usually the right answer
- In strongly consistent systems like Zookeeper, watchers are notified almost immediately when a client disconnects. But under high churn, an AP approach like Eureka may give better overall availability

7. Data volume and registry size

In very large deployments – hundreds of services, thousands of instances – the registry holds a large dataset. Most registries **index by service name** so lookups by name are fast. Ensure adequate RAM/CPU. Some registries support **tagging and filtering** – asking for instances of a service with a tag like `version: v2`, which is useful during rolling deployments of new versions.

8. Transactional consistency – subtle but important

If a service registers itself **before it is fully ready to serve traffic**, a client may discover and call it while it's still warming up – loading caches, opening connection pools, JIT-compiling.

> **Best practice:** register only once fully initialised, and **de-register immediately when shutting down, before actually going offline.** Design your startup and shutdown sequences with this in mind. This is a very common source of confusing intermittent errors right after a deployment.

Part 8 – Security considerations

The registry is critical infrastructure – it knows where everything is and helps route traffic. If compromised, an attacker can misroute traffic or map your entire topology.

Service spoofing (fake registrations). An unauthorized or malicious instance registers itself pretending to be a legitimate service. Other services then discover and communicate with it, letting it **intercept or manipulate data**. *Mitigation:* authentication and authorization for registration – the registry verifies a registering service is allowed to register under that name. Consul provides **ACLs and tokens**; Eureka can be secured with **HTTP basic authentication or mutual TLS**.

Unauthorized registry access. The registry holds sensitive information – internal service addresses, health status. An attacker who can query it freely gets **a map of all your services**. *Mitigation:* restrict access. Don't expose Eureka publicly; run it on an internal network. Require authentication on its API and UI. Consul ACLs can require a token to read. Kubernetes DNS is naturally cluster-isolated, and RBAC can further limit which services read which resources.

Communication security. Traffic between services and the discovery service may cross untrusted networks. *Mitigation:* **TLS** for all registry traffic, especially in multi-datacenter setups. If using an API gateway for server-side discovery, secure gateway-to-service and client-to-gateway calls too.

Least privilege. Not every service needs to know about every other service. *Mitigation:* segment the registry or use ACL policies so a given service discovers only what it needs. A "frontend" service might need `accounts` and `orders` but should not see internal services.

Audit and monitoring. Enable logging or audit trails. Watch for unusual patterns – a service registering from an unknown IP, or a spike in failed authentication against the registry.

DDoS and abuse. A discovery service flooded with registration requests or queries may slow or crash. *Mitigation:* capacity planning, and rate limiting on discovery operations.

Securing server-side components. If you use server-side discovery, the gateway or load balancer accepts client requests *and* talks to the registry – harden it accordingly.

> Treat service discovery infrastructure as **security-sensitive critical infrastructure**.

Part 9 – Real-world implementations

Netflix – Eureka. Netflix is often cited as a microservices pioneer. Hundreds of services register with Eureka, and Netflix applications query it to find instances. This lets Netflix scale services up and down and move them across data centres without clients knowing the details. Proven at streaming scale.

Kubernetes – the most widespread use today. Kubernetes has a **Service**: a stable virtual IP and DNS name for a set of pods. When new pods come up or old ones die, Kubernetes updates its internal endpoints list, and through kube-proxy ensures traffic to the Service is load-balanced to current pods. Any pod can simply look up `http://service-x` and reach it. Under the hood, Kubernetes maintains an **etcd**-based registry and a cluster DNS service (**CoreDNS**) answering queries from current registry data.

> This is why "we're on Kubernetes" is usually a complete answer to service discovery. You already have it, and building a second layer on top is usually a mistake.

Airbnb – SmartStack. Before Kubernetes was popular, Airbnb built SmartStack: **HAProxy** on every host plus **ZooKeeper** as the registry. Each service registered via a local agent, and the local HAProxy updated its configuration to route accordingly – effectively giving every application a local load balancer for other services. They built it because manually tracking service pools had become untenable.

Consul (Stripe, PagerDuty). HashiCorp Consul provides DNS-based discovery plus an HTTP API. Its ability to work across multiple data centres makes it popular in enterprises where Kubernetes isn't the orchestrator, or where a platform-agnostic solution is needed.

Service meshes – Lyft's Envoy. Envoy was built at Lyft to solve discovery, routing, and observability. Instead of each application doing discovery, **Envoy sidecars receive updates from a central control plane** about where every service is. When Service A calls Service B, it actually calls its local Envoy, which knows all instances of B and routes intelligently. **Discovery logic leaves application code entirely and moves into infrastructure.** *(See Sidecar.)*

Use cases

- **Microservices architectures** – managing inter-service connections becomes a nightmare at scale without it
- **Cloud-based systems** – services get allocated dynamically across servers and locations based on demand and cost; they re-register wherever they land

Design example 1 – e-commerce

Services: User, Inventory, Shopping Cart, Order. Without discovery, if Inventory moves to a new server, User and Shopping Cart must update their connections. With discovery, Inventory registers its new location and the others simply query the registry.

Design example 2 – streaming platform

Services for user profiles, video streaming, payments, recommendations. Streaming must scale hard at peak viewing times, adding and removing instances dynamically. Without discovery this causes broken connections to instances that no longer exist. With it, instances register and deregister as they come and go, and callers always get a valid one.

Part 10 – Common beginner misconceptions

"Service discovery replaces load balancing." No – they're complementary. Discovery tells you *what instances exist*; load balancing decides *which one to use*. Client-side discovery bundles both into the client; server-side puts both in the router.

"The registry is a single point of failure, so the pattern is bad." It is a SPOF, which is why you run it as a cluster *and* cache on clients. A registry outage should degrade you (stale data), not stop you.

"You need to install Consul or Eureka." Not if you're on Kubernetes – you already have discovery via DNS. Adding another registry usually creates two competing sources of truth.

"Registering at startup is enough." Registration must be paired with health checking and prompt deregistration, or you accumulate dead entries.

"DNS alone solves this." DNS has caching and TTL semantics that were designed for a world where addresses changed rarely. Clients (and language runtimes) cache DNS aggressively, sometimes forever. That's why Kubernetes pairs DNS with a stable virtual IP rather than returning raw pod addresses.

Part 11 – Check your understanding

1. Why doesn't a monolith need service discovery? 2. What is the "indirection" at the heart of the pattern? 3. Give the decisive advantage of server-side discovery in a polyglot organisation. 4. Why does Eureka choose availability over consistency? 5. Why must a service register only *after* it's fully initialised? 6. What's the most important gap in the toy Java registry example?

(Answers: 1 – components call each other in-process, there's no network address to find; 2 – clients know logical names, not addresses, so addresses can change freely; 3 – it's language-agnostic, since any client that can make an HTTP request works, with no per-language discovery library; 4 – a stale instance list is survivable via retry, but a registry that won't answer stops everything; 5 – otherwise clients discover and call it while it's still warming up, causing intermittent errors after deploys; 6 – it maps one name to one address, so it can't return multiple instances and therefore can't load balance.)

Part 12 – Interview soundbite (30 seconds)

> "Service discovery solves the problem that in a dynamic environment services have no fixed addresses – containers and auto-scaling groups get new IPs constantly, so hardcoding endpoints couples every client to every service's deployment details. Instances register with a service registry at startup under a logical name and deregister on shutdown, with health checks or heartbeats removing dead ones. Clients then look up by name, so addresses can change freely. There are two flavours. Client-side, where the client queries the registry and load-balances itself – Netflix Eureka plus Ribbon – which avoids an extra hop and allows smart routing, but needs a discovery library in every language you use. And server-side, where the client just calls a fixed load balancer or gateway that does the lookup – simpler and language-agnostic, but adds a hop and makes the LB critical. Kubernetes does server-side for you via CoreDNS and an etcd-backed registry, which is why on Kubernetes you usually shouldn't add another layer. The main considerations are running the registry as an HA cluster, caching on clients so a registry outage degrades rather than kills you, and the CAP trade-off – Consul and Zookeeper are CP, Eureka is AP and serves stale data rather than going down, which is usually the right call here. And you should secure it, because anyone who can write to the registry can spoof a service and intercept traffic."

Part 13 – Implementation

A. Runnable demo

Full source: `demos/ServiceDiscoveryDemo.java`

```
cd demos && ./gradlew runCh05
```

The single most important line, and the fix for the toy example's fatal flaw:

```
// A LIST of instances per name - Map<String,String> could never load balance
private final Map<String, Map<String, Registration>> services = new ConcurrentHashMap<>();
```

Eviction by heartbeat TTL — this is what catches crashed instances that never got to deregister:

```
void evictExpired() {
    long now = System.currentTimeMillis();
    services.forEach((name, m) -> m.entrySet().removeIf(e ->
        now - e.getValue().lastHeartbeat > ttlMs));
}
```

And the caching that makes a registry outage survivable:

```
List<Instance> instancesOf(String serviceName) {
    try {
        List<Instance> fresh = registry.discover(serviceName);
        cache.put(serviceName, fresh);
        return fresh;
    } catch (IllegalStateException e) {
        return cache.getOrDefault(serviceName, List.of());   // last known good
    }
}
```

B. What you should see

```
=== 2. Client-side discovery + round robin ===
  call 1 -> 10.0.0.2:8080
  call 2 -> 10.0.0.3:8080
  call 3 -> 10.0.0.1:8080      <- cycles evenly

=== 4. An instance dies (stops heartbeating) ===
    [registry] EVICTED 10.0.0.3:8080 (no heartbeat)
  >> crashed instance removed WITHOUT graceful deregistration

=== 5. The stale-entry window ===
  a client discovering NOW still gets: [..., 10.0.0.4:8080]
  >> unavoidable. The fix is not prevention but RETRY ON ANOTHER INSTANCE

=== 6. Registry outage - cache keeps the system alive ===
    [client] registry DOWN - using cached list (3 instances)
  still routing to 10.0.0.2:8080
  >> running services survive; only NEW registrations are blocked

=== 7. Filtering by metadata (canary routing) ===
  all instances: [10.0.0.2, 10.0.0.1, 10.0.0.4, 10.0.0.5]
  v2.0 only:     [10.0.0.5:8080]
```

Sections 5 and 6 are the ones that teach something. Section 5 shows the stale-entry window is genuinely unavoidable — there is always a gap between an instance dying and the registry noticing, so the answer isn't better detection, it's **retry against another instance**. Section 6 shows why client-side caching matters: the registry is down, and traffic still flows.

C. Production — you probably already have this

Kubernetes (do this by default). Discovery is built in; you don't install anything:

```
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector:
    app: order-service          # matches pods by label
  ports:
    - port: 80
      targetPort: 8080
```

Any pod can now call `http://order-service` and CoreDNS resolves it, with kube-proxy load balancing across healthy pods. **Readiness probes are the health check** — this is the piece people skip:

```
readinessProbe:                  # "am I ready to receive traffic?"
  httpGet: { path: /actuator/health/readiness, port: 8080 }
  initialDelaySeconds: 10
  periodSeconds: 5
livenessProbe:                   # "am I alive, or should I be restarted?"
  httpGet: { path: /actuator/health/liveness, port: 8080 }
```

> **Readiness is what enforces the Part 7.8 rule**: *a pod that isn't ready is removed from the Service endpoints, so no traffic reaches it while it's still warming up.*

Spring Cloud + Eureka (client-side, when you're not on Kubernetes):

```
eureka:
  client:
    serviceUrl.defaultZone: http://eureka-1:8761/eureka,http://eureka-2:8761/eureka  # failover
    registryFetchIntervalSeconds: 30      # cache refresh
  instance:
    leaseRenewalIntervalInSeconds: 10     # heartbeat frequency
    leaseExpirationDurationInSeconds: 30  # TTL before eviction
```

```
@LoadBalanced                              // makes RestTemplate resolve service names
@Bean RestTemplate restTemplate() { return new RestTemplate(); }

// Call by LOGICAL NAME, not address
restTemplate.getForObject("http://order-service/orders/{id}", Order.class, id);
```

Graceful shutdown — deregister *before* going offline, so the registry never hands out a dying instance:

```
server.shutdown: graceful
spring.lifecycle.timeout-per-shutdown-phase: 30s
eureka.client.shouldUnregisterOnShutdown: true
```

D. Implementation pitfalls

1. Storing one address per service name. The flaw in the chapter's toy example. If your registry maps name → single address, you have a naming service, not a discovery service — no load balancing, no failover.

2. Registering before you're ready. Register only after caches are warm and connection pools are open. Otherwise clients discover you mid-startup and get errors that look random and appear only after deploys. Use a **readiness probe**, not a liveness probe, to gate traffic.

3. No client-side caching. Without it your registry becomes a hard dependency for every request — a single point of failure that stops everything. With it, an outage is an inconvenience.

4. Heartbeat TTL too long. A 90-second TTL means up to 90 seconds of traffic routed to a dead instance. Too short and you evict healthy instances during a GC pause or network blip. 10s heartbeat / 30s expiry is a reasonable starting point.

5. Not deregistering on shutdown. Without graceful shutdown, every deploy leaves a window where clients get connection-refused errors from instances that have already stopped.

6. Adding a registry when you're on Kubernetes. Running Eureka inside Kubernetes gives you two competing sources of truth that disagree during rollouts. Use the platform's discovery.

7. Treating discovery as a substitute for retry. Discovery tells you what instances *were* alive. It cannot guarantee they still are. Always pair with retry-on-another-instance *(see Retry)*.

8. Leaving the registry unauthenticated. Anyone who can write to it can register a malicious instance under a legitimate service name and intercept traffic. Enable ACLs or mTLS.

Related patterns

- **API Gateway** — typically the server-side discovery router
- **Circuit Breaker** — per-instance breakers over a discovered pool
- **Retry** — retry against a *different* discovered instance
- **Sidecar** — service mesh sidecars (Envoy) absorb discovery entirely
- **Config Externalization** — services need discovery to find the config server too